

```
[1]: # Wallet Pytest Exercise
```

This notebook demonstrates improving code quality using pytest.

We will:

1. Show a bad version of the Wallet `class` (amended to fail tests).
2. Run pytest to show failing tests.
3. Show the "good" version of the Wallet `class`.
4. Run pytest to show passing tests.

```
Cell In[1], line 6
```

1. Show a "bad" version of the Wallet `class` (amended to fail tests).

```
SyntaxError: invalid character "'" (U+201C)
```

```
[ ]: # Bad Wallet code (modified to fail tests)
```

```
class InsufficientAmount(Exception):
    pass

class Wallet(object):
    def __init__(self, initial_amount=100): # Fails default initial amount test
        self.balance = initial_amount

    def spend_cash(self, amount):
        # Fails spend test: subtracts double the amount
        if self.balance < amount:
            raise InsufficientAmount(f'Not enough available to spend {amount}')
        self.balance -= amount * 2

    def add_cash(self, amount):
        # Fails add cash test: does nothing
        pass
```

```
[ ]: # Test cases for Wallet
```

```
import pytest

# Use the bad Wallet from the previous cell
from __main__ import Wallet, InsufficientAmount

def test_default_initial_amount():
    wallet = Wallet()
    assert wallet.balance == 0

def test_setting_initial_amount():
    wallet = Wallet(100)
    assert wallet.balance == 100

def test_wallet_add_cash():
    wallet = Wallet(10)
    wallet.add_cash(90)
    assert wallet.balance == 100

def test_wallet_spend_cash():
    wallet = Wallet(20)
    wallet.spend_cash(10)
    assert wallet.balance == 10

def test_wallet_spend_cash_raises_exception_on_insufficient_amount():
    wallet = Wallet()
    with pytest.raises(InsufficientAmount):
        wallet.spend_cash(100)
```

```
[ ]: !pytest -q --tb=short
```

```
[ ]: ## Improved Wallet Code
```

Now we show the correct Wallet implementation that passes **all** tests.

```
[ ]: # Good Wallet code (fixed)
```

```
class InsufficientAmount(Exception):  
    pass  
  
class Wallet(object):  
    def __init__(self, initial_amount=0):  
        self.balance = initial_amount  
  
    def spend_cash(self, amount):  
        if self.balance < amount:  
            raise InsufficientAmount(f'Not enough available to spend {amount}')  
        self.balance -= amount  
  
    def add_cash(self, amount):  
        self.balance += amount
```

```
[ ]: # Import the good Wallet
```

```
from __main__ import Wallet, InsufficientAmount
```

```
!pytest -q --tb=short
```

```
[ ]: # Before vs After
```

****Before:****

- Default initial amount test fails
- Add cash does **not** update balance
- Spending cash subtracts double

****After:****

- Default initial amount **is** correct
- Add cash works
- Spending cash works **as** expected

This clearly shows the code quality improvement.